Equality from iteration

A strictly smaller raw presentation of iPRA,
and a six-rule presentation with one uniform evaluation rule

Prepared for Vladimir Reshetnikov

November 11, 2026

Abstract

The current computational-equality presentation of ConciseIPRA separates three evaluation cases: root equality composition, zero iteration, and successor iteration. The first is redundant. A zero-iteration expression provides an identity frame through which two applications of the second case derive the first. Deleting the root case leaves every term, formula, context, substitution operation, and logical inference unchanged. It preserves every derivable sequent and therefore the established bi-interpretation with full first-order intuitionistic primitive recursive arithmetic. A matched, fully validated standalone Python checker decreases from 959 to 954 significant tokens, from 1026 to 1019 AST nodes, and from 3039 to 2996 normalized bytes.

A companion presentation makes zero iteration definitional. Reflexivity and a single successor-evaluation schema then supply all arithmetic computation and equality transport. With the four inherited logical schemas, this gives six displayed rules and just one arithmetic evaluation case; the inherited DI retains its stated syntax-directed test. Its complete checker is 3011 normalized bytes; its 979 tokens and 1047 AST nodes expose the cost of canonical syntax rather than hiding it. Both constructions have whole numerical domains, actual equality, effective proof translations, and both formula round trips. These are local mathematical simplifications; no absolute minimum or new record for the repository's different compact certificate protocols is claimed.

1 Result, baseline, and meaning of simplicity

The comparison is the supplied repository, frozen at commit

`fa9d969919315b97796a1dd288f431cd3e41ffca`.

The relevant project is `Logic/PrimitiveRecursiveArithmetic/ConciseIPRA`. Its README records fifty research packages and a Lean development of the native calculus and its bi-interpretation with iPRA [1]. The immediate baseline is `Research/computational_equality`, whose calculus is formalized in `Lean/ConciseIPRA/NativeEval.lean` [2]. We denote that calculus by $\mathcal{E}_3$, naming its three evaluation cases.

Two conclusions should be distinguished.

1. **A strict improvement on the unchanged raw language.** The calculus $\mathcal{E}_2$ deletes one primitive evaluation case. Its complete checker improves all three measured source coordinates against the exact baseline, with unchanged validation and certificate format.

2. **A simpler arithmetic interface on canonical terms.** The calculus $\mathcal{Z}_1$ uses only reflexivity and one uniform successor evaluation for arithmetic and equality. It has six displayed rule schemas. Its normalizer is part of the trusted syntax and part of every reported checker measurement. This improves the rule structure, but does not dominate $\mathcal{E}_2$ in checker size.

The baseline report calls its calculus "six rules" because Eval is one instruction with three branches. The following table makes the counting convention explicit. The four common logical schemas are Hyp, QE, DE, and the single witness-directed DI.

Quantity	$\mathcal{E}_3$	$\mathcal{E}_2$	$\mathcal{Z}_1$
Serialized proof instruction tags	6	6	6
Rule constructors, separating Eval cases	8	7	6
Arithmetic/equality schemas	4	3	2
Evaluation cases	3	2	1
Supplied equality templates	0	0	0
Zero normalization in substitution	no	no	yes

The actual Lean definition of $\mathcal{E}_3$ has separate constructors `evalId`, `evalZero`, and `evalSucc`. Counting the branches is therefore mathematically meaningful, independent of the choice of proof instruction tags. This convention retains DI as one schema with its explicit mode-selection test; it does not assert that six is an encoding-independent rule count.

2 The complete raw presentation $\mathcal{E}_2$

2.1 Terms, formulas, and substitution

There is one numerical sort. Raw terms are finite trees

$$t ::= x \mid 0 \mid S(t) \mid I(n, a; u.h).$$

The variable u is bound only in the update body h, not in the counter n or initializer a. Bodies can have passive numerical parameters and nested iterators. All substitutions in $\mathcal{E}_2$ are ordinary capture-free substitutions, with no reduction of an iterator.

Native formulas and the designated contradiction are

$$A ::= (s = t) \mid \mathsf{Q}_y A \mid \mathsf{D}_{x,y} A, \qquad \bot_* := (0 = S0).$$

In $\mathsf{D}_{x,y}$ the variables x, y are distinct. A *literal equation* has equality at its outermost formula node. An equation under a binder is not a literal equation. Contexts Γ are finite reusable lists of formulas. The ordinary intuitionistic readings are

$$(\mathsf{Q}_y B(y))^\circ = B^\circ(0) \to \forall y\, B^\circ(y),$$
$$(\mathsf{D}_{x,y} B(x,y))^\circ = \exists x \big(B^\circ(x,0) \to \forall y\, B^\circ(x,y)\big).$$

These readings specify an interpretation, not extra object-level rules.

2.2 Every primitive inference

For a matrix $B(x, y)$ and numerical witness t, put

$$\eta_{B,t}(z) = \begin{cases} z, & B \text{ is a literal equation and } t \equiv 0, \\ 0, & \text{otherwise.} \end{cases}$$

Here $\equiv$ means literal identity of syntax, not provable equality. The entire rule inventory of $\mathcal{E}_2$ is:

$$\frac{A \in \Gamma}{\Gamma \vdash A} \quad \text{Hyp,}$$

$$\frac{}{\Gamma \vdash s = s} \quad \mathsf{Refl,}$$

$$\frac{\Gamma \vdash a = t \qquad \Gamma \vdash I(0, a; u.h) = r}{\Gamma \vdash r = t} \quad \mathsf{E0,}$$

$$\frac{\Gamma \vdash I(c, a; u.h) = t \qquad \Gamma \vdash I(Sc, a; u.h) = r}{\Gamma \vdash r = h[t/u]} \quad \mathsf{ES,}$$

$$\frac{\Gamma \vdash \mathsf{Q}_y B(y) \qquad \Gamma \vdash B(0)}{\Gamma \vdash B(v)} \quad \text{QE,}$$

$$\frac{\Gamma \vdash \mathsf{D}_{x,y} B(x, y) \qquad \Gamma, \mathsf{Q}_y B(b, y) \vdash A}{\Gamma \vdash A} \quad \text{DE,}$$

$$\frac{\Gamma, B(t, \eta_{B,t}(z)) \vdash B(t, Sz)}{\Gamma \vdash \mathsf{D}_{x,y} B(x, y)} \quad \text{DI.}$$

In DE, b is fresh for Γ, A, and the passive parameters of B. In DI, z is fresh for Γ, t, and the passive parameters of B. Displayed local assumptions are discharged. All rule data must be well-scoped terms and formulas.

There are no other primitive logical, equality, successor, computation, induction, or structural axioms. In particular, successor injectivity, separation from zero, and ordinary logical connectives are derived through the established native interpretation. The equality rules require no user-supplied numerical template. Their two premise conclusions determine the result or determine rejection.

The baseline $\mathcal{E}_3$ is exactly these rules plus

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash s = r}{\Gamma \vdash r = t} \quad \text{Root.} \tag{Root}$$

It is this one inference, and only this inference, that is deleted in the raw improvement.

3 The identity-frame argument

Lemma 3.1 (Identity frame)**.** *Suppose a calculus has reflexivity and, for every term s, a term $K(s)$ supporting the inference*

$$\frac{s = t \qquad K(s) = r}{r = t}.$$

Then it derives Root. It does not need $K(s) = s$ as an axiom.

Proof. For given $p : s = t$ and $q : s = r$, reflexivity supplies $K(s) = K(s)$. Apply the displayed inference to q and that reflexivity to get $K(s) = r$. Apply it again to p and the new equation to obtain $r = t$. Both supplied proofs are used once. □

Theorem 3.2 (Root is redundant on raw terms)**.** *For every raw context* Γ *and native formula* A*,*

$$\Gamma \vdash_{\mathcal{E}_2} A \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathcal{E}_3} A.$$

The translations leave every numerical term and every formula unchanged.

Proof. Take $W(s) = I(0, s; u.0)$ in Lemma 3.1. Its displayed inference is precisely $\mathsf{E0}$ with update 0. More explicitly, define

$$\mathsf{R}(p, q) := \mathsf{E0}(p, \mathsf{E0}(q, \mathsf{Refl}(W(s)))) \quad (p : s = t,\ q : s = r).$$

The inner evaluation concludes $W(s) = r$ and the outer one concludes $r = t$. Thus every Root instance of an $\mathcal{E}_3$ proof has an $\mathcal{E}_2$ replacement over exactly the same context. Recursively replace such instances. Every other inference is unchanged. The reverse translation is inclusion, since the remaining rules are baseline rules. □

Corollary 3.3 (Exact proof-tree overhead)**.** *If a baseline proof tree* p *has* N *inference nodes and* k *Root nodes, the displayed compiler produces exactly* $N + 2k$ *inference nodes.*

Proof. Each replaced Root node becomes two $\mathsf{E0}$ nodes and one Refl node; its two premise trees occur once. All other nodes are unchanged. □

This is a node count, not a certificate-byte bound. The additional reflexivity certificate contains the term $W(s)$, whose length depends on the source equation. No claim of constant byte overhead is intended.

The argument is structural: direct equality composition is already present inside the computational rule. Deleting it is justified before any checker is written, and does not use induction, a logical binder, a term-normalization oracle, or an assumed substitution rule on proofs.

4 Arithmetic and equality recovered explicitly

All constructions below are valid over an arbitrary retained context. For $p : s = t$, write

$$\mathrm{Sym}(p) = \mathsf{R}(\mathsf{Refl}(s), p) : t = s.$$

For $p : s = t$ and $q : t = v$, write

$$\mathrm{Trans}(p, q) = \mathsf{R}(q, \mathrm{Sym}(p)) : s = v.$$

These are derived proof constructors, not added primitives.

Lemma 4.1 (The computation equations)**.** *Both original iterator equations have three-node proofs in* $\mathcal{E}_2$*:*

$$I(0, a; u.h) = a, \qquad I(Sc, a; u.h) = h[I(c, a; u.h)/u].$$

Proof. Use the following two evaluations, respectively:

$$\mathsf{E0}(\mathsf{Refl}(a), \mathsf{Refl}(I(0, a; u.h))), \qquad \mathsf{ES}(\mathsf{Refl}(I(c, a; u.h)), \mathsf{Refl}(I(Sc, a; u.h))).$$

□

Lemma 4.2 (Arbitrary numerical-template transport)**.** *From $s = t$ and $h[s/u] = r$, $\mathcal{E}_2$ derives $r = h[t/u]$.*

Proof. Let $C_0 = I(0, s; u.h)$ and $C_1 = I(S0, s; u.h)$. The preceding lemma gives $C_0 = s$. Compose with the first supplied equation to get $C_0 = t$. Separately,

$$\mathsf{ES}(C_0 = s, \mathsf{Refl}(C_1)) : C_1 = h[s/u].$$

Compose with the second supplied equation to get $C_1 = r$. Now $\mathsf{ES}(C_0 = t, C_1 = r)$ concludes $r = h[t/u]$. □

The template h appears in this *derived* construction as an update body. It is not an extra parameter of a primitive equality inference. The same construction permits multiple occurrences of the variable and occurrences under iterator bodies, using ordinary capture-free substitution. Transport in both sides of an equation follows from two applications of the crossed one-template rule.

5 A companion with a single evaluation schema

5.1 Canonical terms and the cost of normalization

Canonical terms have the same constructors $x, 0, S, I$, but an iterator node $I(c, a; u.h)$ is legal only if c is not literally 0 and all its subterms are canonical. For canonical arguments define

$$\mathcal{I}(c, a; u.h) = \begin{cases} a, & c \equiv 0, \\ I(c, a; u.h), & c \not\equiv 0. \end{cases}$$

For raw terms, define bottom-up normalization by

$$\mathrm{nf}(x) = x, \qquad \mathrm{nf}(0) = 0, \qquad \mathrm{nf}(St) = S\,\mathrm{nf}(t),$$
$$\mathrm{nf}(I(c, a; u.h)) = \mathcal{I}(\mathrm{nf}(c), \mathrm{nf}(a); u.\,\mathrm{nf}(h)).$$

In particular, the test is on the *normalized* counter. No successor iteration is evaluated. Term normalization is a finite structural traversal, not unbounded evaluation of a primitive-recursive program. Normalization also traverses bodies beneath their binders.

Write $t\langle\sigma\rangle := \mathrm{nf}(t[\sigma])$ for canonical substitution, and let fnf normalize the terms of a native formula, preserving its formula constructors. These operations are part of the specification of $\mathcal{Z}_1$ and are implemented in its checker.

Lemma 5.1 (Normalization laws)**.** *Normalization is idempotent, commutes with variable renaming, introduces no free variables, and satisfies*

$$\mathrm{nf}\big(\mathrm{nf}(t)[\mathrm{nf} \circ \sigma]\big) = \mathrm{nf}(t[\sigma]). \tag{N}$$

Consequently canonical substitution is associative and has the expected identity law on canonical terms. The analogous identities hold for formulas.

Proof. Induct on the finite term tree. The variable, zero, and successor cases are immediate. At an iterator, first apply the induction hypotheses to its counter, initializer, and body, lifting the substitution through the body binder. If the resulting counter is zero, both sides of (N) reduce to the same normalized initializer; otherwise both sides retain the same iterator node. This also proves the renaming and free-variable claims. Associativity follows by applying (N) to ordinary capture-free substitution composition. Formula induction proves the remaining claims. □

The repository's `ZeroNormalization` module gives formal proofs of these statements, including `nf_subst` and `hsub_comp` [3]. We use that established normalization layer, rather than proposing an unanalysed conversion relation.

5.2 All six rules of $\mathcal{Z}_1$

Formulas are still equality, Q, and D, now with canonical terms. The rules are Hyp, Refl, QE, DE, DI from Section 2, with every numerical instance understood as canonical substitution, together with the following single evaluation rule in place of $\mathsf{E0}$ and the raw ES:

$$\frac{\Gamma \vdash \mathcal{I}(c, a; u.h) = t \qquad \Gamma \vdash I(Sc, a; u.h) = r}{\Gamma \vdash r = h\langle t/u\rangle}. \tag{UE}$$

There is no zero evaluation branch, no root evaluation branch, and no computation axiom. The literal-zero test in DI applies to the canonical witness as supplied. The same freshness requirements apply.

The second premise of UE reveals c, a, h: its left side must be an iterator with a successor counter. The first premise is checked against its canonical predecessor iterate. Thus UE is deterministic and has no additional term-template parameter in its certificate.

Lemma 5.2 (Root equality from a one-step identity iterator)**.** *Define $K(s) = I(S0, s; u.u)$. From $p : s = t$ and $q : s = r$, UE derives $r = t$ by*

$$\mathsf{R}_Z(p, q) = \mathrm{UE}(p, \mathrm{UE}(q, \mathsf{Refl}(K(s)))).$$

Proof. The predecessor iterate of $K(s)$ is $\mathcal{I}(0, s; u.u) = s$. The inner inference concludes $K(s) = r$; the outer concludes $r = t$. This is Lemma 3.1 with a different identity frame. The frame itself is not normalized away. □

Lemma 5.3 (Computation and congruence in $\mathcal{Z}_1$)**.** *UE and Refl derive normalized successor computation, symmetry, transitivity, and transport in every numerical term template.*

Proof. Set $C = \mathcal{I}(c, a; u.h)$ and $B = I(Sc, a; u.h)$. Applying UE to the reflexivities of C and B gives

$$B = h\langle C/u\rangle.$$

Symmetry and transitivity follow from R_Z exactly as before. For a supplied $p : s = t$ and a template h, put $H = I(S0, s; u.h)$. UE gives

$$e_s : H = h\langle s/u\rangle, \qquad e_t : H = h\langle t/u\rangle$$

using respectively $\mathsf{Refl}(s)$ and p as major premises, and $\mathsf{Refl}(H)$ as the minor premise. Then $\mathsf{R}_Z(e_t, e_s)$ gives $h\langle s/u\rangle = h\langle t/u\rangle$. For a second supplied proof $q : h\langle s/u\rangle = r$, another R_Z gives $r = h\langle t/u\rangle$. □

This final transport expansion adds six UE nodes and five Refl nodes; each supplied premise proof occurs once. No logical rule or induction is used in this equality bootstrap.

Proposition 5.4 (Stability of UE)**.** *UE is stable under every canonical numerical substitution.*

Proof. Its minor counter remains successor-headed after substitution. Its major becomes the smart predecessor iterate with substituted fields, by Lemma 5.1, including when the substituted counter becomes literal zero. The same lemma identifies the substituted conclusion with the canonical instance of the new update body. □

This statement concerns UE. The syntax-directed single DI is not preserved rule-for-rule by arbitrary numerical substitution: a nonzero syntax witness can become literal zero. Admissibility of that operation for the full calculus follows from the equivalence below; it is not silently assumed in the argument.

6 Equivalence of the canonical calculus

Let $\mathcal{N}_Z$ be the repository's `NZPrf` calculus. It has canonical syntax, Hyp, successor computation, crossed one-template transport, QE, DE, case introduction for every matrix, and direct equational progress introduction

$$\frac{\Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash \mathsf{Q}_y E(y)}. \tag{QI}$$

Here E is a literal equation and the numerical instances are canonical. Its equivalence to the original native kernel on normalized sequents is proved in `NativeZeroNormal.lean` [3].

Lemma 6.1 (Structural operations needed below)**.** *Injective renaming of variables, weakening, and cut for reusable formula assumptions are admissible in* $\mathcal{Z}_1$*.*

Proof. For renaming, induct on the derivation. Renaming preserves literal zero, formula roots, and the canonical grammar; it commutes with substitution. Thus DI retains its original mode. The other rules are equally stable. For weakening and cut, rename eigenvariables away from newly inserted data, then induct on the derivation. Cut replaces a use of the cut hypothesis by its supplied proof. No numerical term is changed, so DI's syntactic test remains unchanged. UE has no discharged assumption; its premise transformations are immediate. The local assumptions of DE and DI are retained when applying the induction hypothesis. This proves the needed structural operations without numerical substitution into proofs. □

Lemma 6.2 (Both introductions of $\mathcal{N}_Z$ are derivable)**.** *The unrestricted case introduction and* (QI) *are admissible in* $\mathcal{Z}_1$*.*

Proof. Case introduction. Only a literal equation matrix $E(x, y)$ at witness 0 is missing. Put

$$Z = I(S0, 0; u.0).$$

This is canonical and is not literally zero. UE on two reflexivities proves $Z = 0$. Suppose the desired case premise is $\Gamma, E(0, 0) \vdash E(0, Sz)$. Under $\Gamma, E(Z, 0)$, equational-template transport along $Z = 0$ gives $E(0, 0)$. Cut with the given premise yields $E(0, Sz)$. Transport along $0 = Z$ gives $E(Z, Sz)$. DI at the nonzero-syntax witness Z uses its case mode and concludes the original $\mathsf{D}_{x,y}E(x, y)$. Only

transport in literal equations was needed; transport through compound native formulas was not used.

Equation progress. Given $\Gamma, E(z) \vdash E(Sz)$, introduce $\mathsf{D}_{x,y}E(y)$ with the unused witness variable x and witness 0. DI is in progress mode. Eliminating this D opens exactly $\mathsf{Q}_y E(y)$, independent of the fresh witness. Hyp and DE therefore give $\mathsf{Q}_y E(y)$ itself. □

The old raw zero alias $I(0, 0; u.0)$ would be invalid as a nonliteral canonical witness: it normalizes to zero. The surviving one-step alias Z is essential to the preceding proof.

Theorem 6.3 (Exact canonical equivalence)**.** *For every canonical context* Γ *and canonical native formula* A*,*

$$\Gamma \vdash_{\mathcal{Z}_1} A \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathcal{N}_Z} A.$$

Proof. First normalize any raw numerical rule parameters admitted by the Lean definition of `NZPrf`. The normalization/substitution identities leave its canonical premises and conclusions unchanged, so we may use canonical data throughout the comparison. From right to left, successor computation and crossed transport are derived by Lemma 5.3; the preceding lemma supplies both introductions. The remaining rules are unchanged. Translate proofs recursively, using the explicit constructions just given.

From left to right, reflexivity is already derivable in $\mathcal{N}_Z$. For UE, successor computation gives

$$I(Sc, a; u.h) = h\langle \mathcal{I}(c, a; u.h)/u \rangle.$$

Transport along the major equality and compose with the reversed minor equality to obtain $r = h\langle t/u \rangle$. DI in case mode is the existing case rule. For a progress-mode DI, the matrix is an equation and the witness is zero. Apply QI to $E(0, y)$. Under the case assumption $E(0, 0)$, QE then gives $E(0, Sz)$; the existing case introduction at witness zero concludes $\mathsf{D}_{x,y}E(x, y)$. Hyp, QE, and DE are unchanged. □

Corollary 6.4 (Raw-to-canonical comparison)**.** *For every raw native sequent,*

$$\Gamma \vdash_{\mathcal{N}} A \quad \Longleftrightarrow \quad \mathrm{fnf}(\Gamma) \vdash_{\mathcal{Z}_1} \mathrm{fnf}(A).$$

Proof. Compose Theorem 6.3 with the repository theorem `nzprf_iff_nprf` [3]. □

7 Bi-interpretability with full first-order iPRA

7.1 The exact inherited bridge

The target iPRA is intuitionistic first-order logic with equality, with one function symbol for each fixed primitive-recursive definition, its defining equations, successor injectivity, $Sx = 0 \to \bot$, and induction for quantifier-free formulas with parameters. This is the theory deeply embedded as `PRAAx` in the supplied project. No classical excluded-middle schema for arbitrary formulas is added.

The repository proves a stronger statement than equiconsistency. Its intermediate theory T_{eq} uses the scalar iterator language, ordinary intuitionistic first-order logic with equality, the two iterator equations, coverage

$$\forall x\, (x = 0 \vee \exists y\, x = Sy),$$

separation $0 = S0 \to \bot$, and literal-equation induction with side assumptions. The formalized bridge establishes:

1. mutually inverse interpretations, up to provable formula equivalence, between the native calculus $\mathcal{N}$ and T_{eq};
2. term interpretations between T_{eq} and iPRA with whole numerical domains, actual equality, and composites provably equal to the original terms;
3. the resulting native/iPRA equivalences on arbitrary contexts and both formula round trips.

The relevant named results are `biInterp`, `native_iff_pra`, `pra_iff_native`, `native_roundTrip`, `pra_native_roundTrip`, and `native_inconsistent_iff` [4].

The arithmetic direction compiles each fixed primitive-recursive definition using a derived predecessor, equality reflection, pairing, and iteration. The reverse direction interprets each fixed iterator body by primitive recursion. No uniform total evaluator over arbitrary program descriptions is postulated. The archive includes the five upstream written bridge components, together with frozen Lean sources, so the inherited part of the result can be inspected separately from the new reduction.

7.2 Why this gives both composites

Theorem 7.1 (Bi-interpretation and equiconsistency)**.** *Both $\mathcal{E}_2$ and $\mathcal{Z}_1$ are bi-interpretable with iPRA, with whole numerical domains and actual equality. Both are equiconsistent with iPRA.*

Proof. For $\mathcal{E}_2$, Theorem 3.2 gives exactly the same sequents as $\mathcal{E}_3$. The theorem `eprf_iff_nprf` then identifies $\mathcal{E}_3$ with $\mathcal{N}$. Consequently the established interpretations and both formula composites apply unchanged. This is stronger than a new mutual-interpretability argument: no formula translation is altered.

For $\mathcal{Z}_1$, use canonical inclusion j into raw syntax and the normalization map fnf back. On canonical formulas, $\mathrm{fnf}(j(A)) = A$ literally. In the native calculus a raw term satisfies $t = \mathrm{nf}(t)$, and every raw formula is interderivable with $\mathrm{fnf}(A)$. These are the existing normalization round-trip theorems [3]. Inclusion also respects canonical substitution up to provable equality:

$$j(t\langle\sigma\rangle) = j(t)[j \circ \sigma] \quad \text{is provable in the raw theory.}$$

This is the substitution-coherence obligation for the interpretation, not an assertion of literal syntactic equality. It follows from `nf_correct` and `inclusion_subst_coherent`. The latter round trip includes reductions under iterator binders: its proof uses equation induction to obtain iterator-body congruence. It is not inferred merely from free-variable congruence.

Corollary 6.4 and Theorem 6.3 therefore make j and fnf inverse interpretations up to provability. Compose them with the established native/iPRA bi-interpretation. No translation restricts the numerical domain, replaces equality by a quotient relation, or introduces additional elements. The numerical identity graph remains $x = y$ in both composites.

Finally $\mathrm{fnf}(0 = S0) = 0 = S0$. The native contradiction corresponds to ordinary $\bot$ under the established bridge, so consistency is equivalent in both directions. □

The new mathematical proofs in this paper are written proofs supported by executable certificate checks. They have not been compiled as a new Lean development. The frozen upstream Lean sources are reference evidence for the inherited results, not a claim that the new checker or the new calculus has received proof-assistant verification.

8 Sharpness of the raw deletion

The reduction does not assert absolute minimality. There is, however, a precise irredundancy statement for the remaining raw arithmetic interface.

Proposition 8.1 (Neither remaining evaluation case can be deleted)**.** *With all the other $\mathcal{E}_2$ rules unchanged, deleting* E0 *loses zero computation, and deleting* ES *loses successor computation.*

Proof. Use standard natural numbers with standard $0, S$, actual equality, and the stated ordinary meanings of Q, D.

If E0 is omitted, interpret

$$I(n, a; u.h) = h^n(Sa).$$

Successor evaluation is valid, as are reflexivity and all the logical rules. For the progress branch of DI, ordinary induction on standard natural numbers suffices; for its case branch, numerical coverage suffices. But $I(0, 0; u.0) = 0$ is false, since the left side is 1. Thus the missing zero equation is not derivable.

If ES is omitted, interpret $I(n, a; u.h) = a$ for every counter and update. Zero evaluation and every unchanged rule remain valid. Yet the required successor equation $I(S0, 0; u.Su) = S(I(0, 0; u.Su))$ reads $0 = 1$ and is false. □

Proposition 8.2 (An atomic seed is necessary)**.** *Deleting Refl from $\mathcal{E}_2$ leaves no derivable equation over an equation-free context; in particular, no equation is provable from the empty context.*

Proof. Induct on a derivation. Hyp cannot produce an equation from such a context. Either evaluation case would require an equation premise. A QE with an equation conclusion has an equation matrix, so it also requires an equation as its base premise. DE adds only a Q-headed assumption, so its local context is still equation-free. DI concludes a D-headed formula. Each possible last inference is therefore excluded by the induction hypothesis. □

These propositions concern deletion of these exact rules. They do not exclude another arithmetic encoding, a different logic, or a smaller certificate implementation.

9 Complete checker and certificate contract

The scripts in `code/` are standalone Python modules. Each reads one JSON object from standard input, validates it, and prints `ACCEPT` only after checking its asserted conclusion. The complete outer contract is

```
{"variables": n, "context": [A0, ...], "goal": A, "proof": p}
```

There are exactly these four fields. The nonnegative integer n is the number of ambient numerical variables. A closed theorem has $n = 0$ and an empty context. A nonempty context asserts the corresponding sequent. Boolean and floating-point indices are rejected.

9.1 Syntax encoding

The representation uses de Bruijn indices. The numerical and formula constructors are:

JSON syntax	Meaning
`["v", i]`	variable with index i
`["z"]`	zero
`["s", t]`	successor
`["i", c, a, h]`	iterator; only h gains one bound variable
`["=", s, t]`	equation
`["q", B]`	Q; body gains one variable
`["d", B]`	D; body gains two variables

In a D body, index zero is y and index one is x. The canonical checker rejects any supplied iterator whose counter is literally zero, including such iterators nested inside bodies. It does not silently erase malformed external syntax before validation. Internally, substitution rebuilds terms bottom-up with the smart constructor; a well-scoped substitution may legitimately create a zero counter and contract it.

9.2 Proof instructions

Instruction	Checked action
`["hyp", i]`	Select a retained assumption.
`["refl", t]`	Validate t, then infer $t = t$.
`["ev", p, q]`	Infer both equations and apply a permitted evaluation case.
`["qe", t, p, q]`	Validate t; infer a Q major and verify the zero instance.
`["de", A, p, q]`	Validate the intended outer conclusion A; open a fresh witness in the minor branch and check its shifted conclusion.
`["di", B, t, p]`	Validate the matrix and witness; construct the required local assumption and check the successor instance.

Contexts are immutable across recursive branch calls. The fresh-variable conditions are enforced by shifting old contexts and by checking the outer conclusion at its original scope. The top-level goal and every certificate-supplied term or formula are validated. Internally inferred formulas are well-formed by induction on accepted proof instructions; there is no user-supplied formula oracle or uncounted proof library.

In raw Eval, the left side of the minor equation must be an iterator. A literal zero counter selects E0; a successor counter selects ES; all other shapes reject. A root match by itself no longer authorizes an inference. In canonical Eval the minor must have a successor counter; its predecessor iterate is reconstructed with the smart constructor. The source equality is compared syntactically.

The arithmetic tags are inferred from well-formed formula and term lengths in a few local destructuring operations. Those operations are safe only because the constructor-validation invariant is established first. The checker does not accept arbitrary Python objects or evaluate Python expressions from a certificate.

10 Measurements and executable evidence

10.1 Matched size comparison

Measurements use CPython 3.12.14. The normalized source is `ast.unparse(ast.parse(source)) + newline`. Significant tokens exclude comments, encoding, end-marker, indentation, dedentation, and newline tokens. AST size is the number of objects visited by `ast.walk`. Bytes are UTF-8 source bytes. Gzip is level 9 with deterministic timestamp zero, on the normalized source.

Complete checker	Tokens	AST nodes	Bytes	Gzip-9
Upstream raw $\mathcal{E}_3$	959	1026	3039	966
Raw $\mathcal{E}_2$	954	1019	2996	955
Canonical, with Root retained	987	1059	3064	982
Canonical $\mathcal{Z}_1$	979	1047	3011	965

The first two rows have exactly the same syntax validator, substitution, logical rules, JSON envelope, and rejection policy. Their difference is the proved deletion of Root: 5 fewer tokens, 7 fewer AST nodes, and 43 fewer normalized bytes (about 1.4%). Gzip improves by 11 bytes; it is reported as information, not as the optimization objective.

The last two rows use the same canonical syntax and normalizing substitution. Deleting Root there saves 8 tokens, 12 AST nodes, and 53 normalized bytes. Comparing $\mathcal{Z}_1$ directly with the raw baseline gives 28 fewer bytes but 20 more tokens and 21 more AST nodes. This is a tradeoff, not coordinatewise domination.

The repository's 516-byte compact checker is a different guarded certificate protocol. The present scripts validate the full Eq/Q/D public language and all externally supplied rule data. No global compact record is claimed. Likewise, the six-rule `initializer_equality/report.tex` uses the same zero-normal scalar grammar, but a different checker implementation [5]. Its arithmetic interface is successor computation plus a two-case Seed rule; the mathematical comparison with $\mathcal{Z}_1$ is

$$\mathrm{IS} + \mathrm{Seed} \quad \longrightarrow \quad \mathsf{Refl} + \mathrm{UE}.$$

We do not present an unmatched byte comparison as evidence for that particular change.

10.2 Verification boundary

The archive provides executable derivations of the new equality bootstraps, computation, numerical-template transport, and representative logical rules; rejected malformed and invalid certificates; normalization and binding checks; a compiler from the baseline raw proof format into $\mathcal{E}_2$; and reproducible source measurements. The verification log and structured results record exactly what was executed. Tests exercise nontrivial rule interactions and attacks on scope and syntax; they are not substitutes for the mathematical soundness and completeness proofs.

The delivered run passed 460 standalone process checks, covering the readable and normalized modules in ordinary and optimized Python modes. The raw family has 38 positive and 25 negative cases; the canonical family has 24 positive and 27 negative cases. All 62 positive cases have checked end-sequent translation round trips. Fourteen raw inputs are the upstream computational-equality examples. In addition, 1280 structural normalization checks and 320 independent numerical substitution checks passed, with zero unexpected outcomes.

Both executable proof compilers eliminate Root within a fixed syntax interface. The canonical compiler compares its canonical control with $\mathcal{Z}_1$; it is not an implementation of the raw-to-canonical translation of Corollary 6.4. That translation is established by the mathematical equivalence and the inherited normalization theorem.

Run from the extracted archive directory:

```
python3 reproduce.py
pdflatex -interaction=nonstopmode -halt-on-error report.tex
pdflatex -interaction=nonstopmode -halt-on-error report.tex
```

Python verification and measurements require no network or third-party Python package. PDF rebuilding requires a normal LaTeX installation with the packages listed in the source. Resource exhaustion is a rejection or failed run, never an acceptance. As with any finite implementation, the completeness theorem concerns the finite proof system with unbounded available resources, not the machine's fixed stack or memory limits.

11 Further research questions

1. **Formalize the new comparison.** On raw syntax, deleting `evalId` should require only the two-step frame lemma and a derivation induction. For $\mathcal{Z}_1$, reuse `ZeroNormalization` and `NativeZeroNormal`, then formalize the canonical zero-alias argument for the merged DI. This would verify the new theorem, beyond the already verified inherited bridge.
2. **Transfer the arithmetic reduction to the compact protocol.** Can the identity-frame principle improve the smallest guarded certificate encodings while retaining their exact public-language and malformed-input contracts? The current measurements do not answer this.
3. **A uniform logical introduction.** UE is substitution-stable, whereas the single witness-directed DI is only admissibly stable after translation. Can one remove that syntactic test without enlarging the complete checker or strengthening induction beyond iPRA?
4. **A genuinely smaller arithmetic basis.** The two remaining raw evaluation cases are deletion-irredundant, but this is not a lower bound across term languages. An alternative basis must account for initializer freedom, equality reflection, and both domain round trips.
5. **Proof size as a secondary cost.** Raw Root elimination adds exactly two inference nodes per occurrence, while potentially copying large source terms into reflexivity certificates. Can sharing reduce that cost without making validation more complicated than the rule it removed?

A The complete raw checker

```
import json
import sys


def require(condition):
    if not condition:
        raise ValueError('Invalid certificate')


def term(a, n):
    match a:
        case ['v', i]:
```

```
            require(type(i) is int and 0 <= i < n)
        case ['z']:
            pass
        case ['s', b]:
            term(b, n)
        case ['i', c, b, h]:
            term(c, n)
            term(b, n)
            term(h, n + 1)
        case _:
            raise ValueError('Invalid term')


def formula(a, n):
    match a:
        case ['=', s, t]:
            term(s, n)
            term(t, n)
        case ['q', b]:
            formula(b, n + 1)
        case ['d', b]:
            formula(b, n + 2)
        case _:
            raise ValueError('Invalid formula')


def walk(a, f, d=0):
    if a[0] == 'v':
        return f(a[1], d)
    return [a[0]] + [walk(b, f, d + ({'i': [0, 0, 1], 'q': [1], 'd': [2]}.get(a[0], [0] * len(a)
    ))[j])
                     for j, b in enumerate(a[1:])]


def shift(a, k=1):
    return walk(a, lambda i, d: ['v', i + k if i >= d else i])


def sub(a, t, drop=1):
    return walk(a, lambda i, d: ['v', i] if i < d else shift(t, d) if i == d else ['v', i - drop
    ])


def infer(p, g, n):
    match p:
        case ['hyp', i]:
            require(type(i) is int and 0 <= i < len(g))
            return g[i]
        case ['refl', a]:
            term(a, n)
            return ['=', a, a]
        case ['qe', t, p, q]:
            term(t, n)
            a = infer(p, g, n)
            require(a[0] == 'q')
            require(infer(q, g, n) == sub(a[1], ['z']))
            return sub(a[1], t)
        case ['de', a, p, q]:
            formula(a, n)
```

```
            b = infer(p, g, n)
            require(b[0] == 'd')
            require(infer(q, [['q', b[1]]] + [shift(x) for x in g], n + 1) == shift(a))
            return a
        case ['di', b, t, p]:
            formula(b, n + 2)
            term(t, n)
            a = sub(['q', b], t)[1]
            h = a if a[0] == '=' and t == ['z'] else shift(sub(a, ['z']))
            require(infer(p, [h] + [shift(x) for x in g], n + 1) == sub(a, ['s', ['v', 0]], 0))
            return ['d', b]
        case ['ev', p, q]:
            _, s, t = infer(p, g, n)
            _, b, r = infer(q, g, n)
            _, c, a, h = b
            if c != ['z']:
                require(c[0] == 's')
                a, t = ['i', c[1], a, h], sub(h, t)
            require(a == s)
            return ['=', r, t]
        case _:
            raise ValueError('Invalid proof')


def check(c):
    require(type(c) is dict and set(c) == {'variables', 'context', 'goal', 'proof'})
    n, g, a, p = (c[k] for k in ['variables', 'context', 'goal', 'proof'])
    require(type(n) is int and n >= 0 and type(g) is list)
    for b in g:
        formula(b, n)
    formula(a, n)
    require(infer(p, g, n) == a)


if __name__ == '__main__':
    try:
        check(json.load(sys.stdin))
    except (ValueError, TypeError, IndexError, KeyError, RecursionError):
        sys.exit(1)
    print('ACCEPT')
```

References

[1] Vladimir Reshetnikov, *ProveIt: Concise presentations of intuitionistic primitive recursive arithmetic*, README at commit `fa9d969919315b97796a1dd288f431cd3e41ffca`. Pinned project directory.

[2] Same repository, *Computational equality: six rules for iPRA*, `Research/computational_equality/report.tex`, with `code/checker.py`; and `NativeEval.lean`.

[3] Same repository, `ZeroNormalization.lean` and `NativeZeroNormal.lean`.

[4] Same repository, `NativeBiInterpretation.lean`, `NativeRoundTrip.lean`, `BiInterpretation.lean`; and `Research/certified_zero/reference/`: `inherited_choice_basis.tex`, `inherited_ordinary_bridge.tex`, `scalar_predecessor.tex`, `equality_reflection.tex`, `primitive_recursion.tex`.

[5] Same repository, *Initializer equality*, `Research/initializer_equality/report.tex`, especially its explicit zero-normal scalar syntax and six-rule calculus.